

When do I use a dot? When do I use a function?

General Rule

- **Use dot (.)** → when you are calling a **method that belongs to the object** (the object “knows how to do it”).
- **Use function name (like `len(...)`)** → when you are calling a **built-in function** in Python that works on many data types.

1. Lists

```
lst = [10, 20, 30]

# Methods (use dot)
lst.append(40)    # add one element
lst.remove(20)    # remove first occurrence of 20
lst.sort()        # sort in place

# Functions (no dot)
print(len(lst))   # number of elements
print(max(lst))   # maximum element
print(sum(lst))   # sum of all elements
```

2. Sets

```
s = {1, 2, 3}

# Methods (dot)
s.add(4)          # add an element
s.remove(2)       # remove element
s.clear()         # empty the set

# Functions (no dot)
print(len(s))     # size of set
print(min(s))     # smallest element
print(max(s))     # largest element
```

3. Dictionaries

```
d = {"a": 1, "b": 2}

# Methods (dot)
print(d.keys())   # get all keys
print(d.values()) # get all values
print(d.items())  # get key-value pairs
d.update({"c": 3}) # add/update

# Functions (no dot)
print(len(d))     # number of key-value pairs
print(type(d))    # type of variable
```

4. Strings

```
s = "hello"

# Methods (dot)
print(s.upper())      # 'HELLO'
print(s.lower())      # 'hello'
print(s.replace("h", "H")) # 'Hello'

# Functions (no dot)
print(len(s))         # 5
print(type(s))        # <class 'str'>
```

5. Variables (general)

```
x = 42
y = "Python"

# Functions
print(type(x))      # <class 'int'>
print(id(x))        # memory ID
print(isinstance(x, int)) # check type
```

Numbers, booleans, and floats usually **don't have many methods** — most work is done with built-in functions or operators.

Memory Trick (For Students)

- If you say: **“Hey OBJECT, do this!”** → Use **dot** (`lst.append(...)`, `s.add(...)`, `s.upper()`)
- If you say: **“Hey PYTHON, tell me something about this object!”** → Use **function** (`len(lst)`, `type(s)`, `max(set)`)

Student Questions & Short Answers

Q1. Why does `len(lst)` work but not `lst.len()`?

Because `len` is a **built-in function** in Python, not a method of the list.

Q2. How can I know if it's a method or a function?

Type the variable, then dot (`mylist.`) and press **Tab** in most Python editors → it shows methods. If it's not there, it's probably a built-in function.

Q3. Why does Python separate them?

Methods are specific to that data type (only lists know `append`), but functions like `len()` work on many types (lists, strings, sets, dictionaries).